

Reflection: The Shift from "Model" to "Harness" -- How OpenAI's Agents Altered My View of AI and My Career

Course: Agentic AI for Business and FinTech (SEEM5660)

Date: September 21, 2026

Task 2: Reflection (10 points)

The Event

On September 10, 2026, OpenAI launched the Agents API in public beta. On the surface, it is a developer product: a managed platform that packages the open-source Codex harness into an API for orchestrating long-running, multi-step tasks -- with automatic context compaction, tool search, programmatic tool calling, multi-agent coordination, and hosted sandboxes for code execution. Developers can use an OpenAI-hosted environment, their own infrastructure, or partner environments, with no separate API fee beyond model tokens and tools consumed.

But the significance of this launch goes far beyond a new endpoint. As one industry analyst put it: "The difficult part of production agents is increasingly everything *around* the model -- state, tools, retries, observability, and safe execution."

OpenAI is turning that surrounding layer into a product. The competitive question is no longer "Which model scores highest on benchmarks?" but "Which system can finish the work reliably and leave evidence that it did so properly?"

This single event reframed how I think about AI, my coursework, and my career.

Before: I Thought the Model Was the Product

Like most students entering a FinTech program, I had a straightforward mental model of how AI creates value: a powerful model sits at the center, you send it a prompt, it returns an answer, and the quality of that answer determines the value. The model is the product; everything else is plumbing.

This view shaped my learning priorities. I focused on understanding model capabilities -- what GPT, Claude, and DeepSeek can do, what their limitations are, how to write better prompts. I treated frameworks like LangChain as convenience wrappers -- tools that make it slightly easier to call models, but not the core skill.

Even when I started this homework -- building a LangChain chain to process supermarket receipt images with DeepSeek's vision model -- I thought the "hard part" was getting the model to read receipts accurately. The chain was just glue code.

The Shift: The Harness **Is** the Product

The Agents API launch forced me to confront a contradiction in my own thinking. If the model were truly the product, why would OpenAI -- the company that **makes** the most powerful models -- invest in productizing the layer **around** the model? The answer is stated plainly in their announcement: because state management, tool orchestration, retries, observability, and safe execution are where production reliability is won or lost. The model generates text; the harness turns that text into completed work.

This clicked for me because of what I experienced firsthand in this homework.

My Homework Was a Harness

When I implemented `build_chain()` and `answer_queries()` in `hw1.py`, I was not merely calling a vision model. I was building a miniature harness:

- * State management -- I encoded receipt images as base64 data URLs, managed a list of multimodal messages, and tracked which receipts had been processed.
- * Tool orchestration -- I used `chain.batch()` to process seven receipts in parallel, coordinating independent calls rather than running them sequentially.
- * Output reliability -- The model's "thinking mode" turned out to be incompatible with `with_structured_output` (function calling), so I had to pivot to a prompt-based JSON approach with regex fallback parsing -- handling failure modes that the model itself could not resolve.
- * Error handling -- I added code-fence stripping, JSON regex extraction, and fallback logic to ensure that even if a single receipt's response was malformed, the aggregation would not crash the entire pipeline.
- * Observability -- I instructed the model to list every discount line in a `discount_lines` array, creating a self-audit trail so that I (and any future reviewer) could verify **why** the model arrived at a particular total.
- * Correctness verification -- The runner code I was given (and told not to modify) compares my responses against `ground_truth.json`, enforcing that the final output contains **exactly one** numeric amount -- a constraint on the harness, not the model.

The model (DeepSeek's `deepseek-v4-flash-vision-exp`) was a component. The **harness** -- the chain I designed, the prompt I engineered, the parsing logic I wrote, the failure modes I anticipated -- was what actually produced the correct answer. And OpenAI just announced that they are building a platform to productize exactly this kind of work.

The realization was jarring: the skill I was practicing for a homework assignment is the skill the most valuable company in AI has decided to turn into a managed API.

The line between "academic exercise" and "industry-level engineering" had just collapsed.

What This Means for Business and FinTech

The implications extend well beyond my homework. If the harness -- not the model -- is where production value accrues, then the competitive dynamics in every AI-powered industry shift:

Models Become Commodities; Harnesses Become Moats

When OpenAI, DeepSeek, Google, and Anthropic all offer capable models through APIs, the model becomes a commodity input. What differentiates a financial institution's AI capabilities is not *which* model it uses, but *how well* it orchestrates that model across thousands of documents, transactions, and customer interactions. A bank that builds a superior harness -- one that reliably processes loan applications, detects fraud, and manages customer inquiries with proper state handling, retries, and audit trails -- will outperform one that simply calls the "best" model.

This means the value in FinTech AI is shifting from model selection to system design: how you chain prompts, how you manage context across long workflows, how you handle edge cases, how you make agents fail safely, and how you produce evidence that the work was done correctly.

The "Model Score" Era Is Ending

For the past two years, the AI conversation has been dominated by benchmark scores -- MMLU, HumanEval, SWE-Bench. The Agents API signals that this era is ending. The new question is not "What does the model score?" but "Can the system finish the job?" A model that scores 95% on a benchmark but drops context after three tool calls is less valuable in production than a model that scores 85% but is wrapped in a harness that manages state, retries, and context compaction.

For FinTech specifically, this means that the metrics that matter are not accuracy on benchmarks, but reliability on real workflows: Did the agent process all 7 receipts correctly? Did it handle the receipt where the discount was listed in Chinese characters? Did it produce exactly one number, or did it hallucinate a second amount? These are harness questions, not model questions.

How This Changed My Career Plan

Before: Model-Centric Career Path

My original plan was to graduate with a FinTech degree, learn enough Python and basic ML to be dangerous, and join a financial institution where I would *use* AI

tools -- calling APIs, running models, generating reports. I saw AI as a skill to acquire and apply, like learning a new programming language. My career ceiling was "person who knows how to use AI well in finance."

After: Harness-Builder Career Path

The Agents API announcement made me realize that career path is already being commoditized. If OpenAI is turning the orchestration layer into a managed API, then "person who calls models" is not a durable career -- it's a task that will be automated by the very platforms that are being built today.

Instead, I see three career directions that are **not** being commoditized, and I am repositioning toward them:

- * Harness Architect for Financial Systems. The harness that OpenAI is productizing is generic -- it works for any domain. But financial systems have domain-specific constraints: regulatory compliance, audit trails, transactional integrity, latency requirements, and risk controls. Building a harness that satisfies **both** the general orchestration patterns **and** the specific constraints of finance is a skill that a generic API cannot fully encapsulate. I want to be the person who designs agentic chains for loan underwriting, fraud detection, and compliance monitoring -- not the person who calls the API, but the person who **architects the system around it**.

- * Agent Reliability Engineer. The Agents API announcement emphasized "evidence that the work was done properly." In finance, this is not just best practice -- it is law. Every automated decision in a regulated financial institution must be auditable. A new role is emerging: engineers who specialize in testing, monitoring, and certifying that AI agent systems produce correct, explainable, and compliant outputs. My experience in this homework -- where I had to debug why receipt2 was \$1.00 short (missed discount line) and receipt7 was \$11.00 short (model skipped multiple discount lines) -- is exactly the kind of failure analysis that this role requires. The harness can break in subtle ways, and someone needs to find and fix those breaks before they reach production.

- * FinTech AI Infrastructure Strategist. OpenAI's Agents API is one harness. Anthropic will build another. DeepSeek may build one for the Chinese ecosystem. Financial institutions will need people who can evaluate, compare, and integrate these platforms -- who understand the trade-offs between hosted vs. self-managed sandboxes, between OpenAI's ecosystem and DeepSeek's, between the speed of managed APIs and the control of custom orchestration. This is a strategic, not merely technical, role -- and it sits at the intersection of business judgment and engineering literacy, which is exactly where a FinTech degree plus hands-on chain-building experience positions me.

What I Am Doing Differently Starting Now

The Agents API event did not just change my **thinking** -- it changed my **actions**:

- * I am treating every LangChain assignment as harness practice, not just homework. The chain I built for this homework -- with its parallel batch processing, JSON parsing fallback, self-auditing prompt design, and ground-truth verification -- is a microcosm of what I will build in industry. I am keeping this code and studying it as a pattern.
- * I am learning agent observability. The OpenAI announcement emphasized tracing, tool-call inspection, and failure recovery. I am studying how to instrument chains so that when (not if) they fail, I can diagnose **where** in the harness the failure occurred -- was it the prompt? the parsing? the model? the aggregation?
- * I am reading the OpenAI Agents API documentation and building a prototype. Not because I need to use it today, but because understanding what the industry leader has productized tells me what skills are becoming standard and what gaps remain for domain-specific (FinTech) applications.
- * I am shifting my course selection. I will prioritize courses in distributed systems, reliability engineering, and AI governance over additional ML theory courses. The theory is valuable, but the harness-building skills are more immediately marketable and harder to automate.

Conclusion

On September 10, 2026, OpenAI did not just launch an API. It declared that the next phase of AI competition has moved outward from the model -- into harnesses, infrastructure, and deployment. The model generates; the harness **delivers**.

I built a small harness for this homework. It processes seven supermarket receipts and produces two numbers. It is simple, but it taught me the lesson that OpenAI just productized for the world: the value is not in the answer the model gives, but in the system that reliably produces, verifies, and delivers that answer.

That lesson has redirected my career. I am no longer aiming to be someone who **uses** AI in finance. I am aiming to be someone who **builds the systems that make AI reliable enough to use in finance.** The Agents API confirmed that this is not a niche -- it is the emerging center of the industry. And I would rather be at the center, building the harness, than at the edge, calling the model.

Submitted as Task 2 reflection for FTEC5660 / SEEM5660 Individual Homework 01.